

7. Implement a stack using arrays in C++, and perform push, pop, and display operations.

```
#include <iostream>
using namespace std;

#define MAX 5 // Maximum size of stack

int stack[MAX];
int top = -1; // Stack is empty initially

// Function to add an element into the stack
void push(int value) {
    if (top == MAX - 1) {
        cout << "Stack Overflow! Cannot push element.\n";
    } else {
        top++;
        stack[top] = value;
        cout << value << " pushed into stack.\n";
    }
}

// Function to remove an element from the stack
void pop() {
    if (top == -1) {
        cout << "Stack Underflow! Stack is empty.\n";
    } else {
        cout << stack[top] << " popped from stack.\n";
        top--;
    }
}

// Function to display stack elements
void display() {
    if (top == -1) {
        cout << "Stack is empty.\n";
    } else {
        cout << "Stack elements are:\n";
        for (int i = top; i >= 0; i--) {
```

```
        cout << stack[i] << endl;
    }
}
}
```

// Main function

```
int main() {
    int choice, value;

    do {
        cout << "\n--- Stack Menu ---\n";
        cout << "1. Push\n";
        cout << "2. Pop\n";
        cout << "3. Display\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter value to push: ";
                cin >> value;
                push(value);
                break;

            case 2:
                pop();
                break;

            case 3:
                display();
                break;

            case 4:
                cout << "Exiting program.\n";
                break;

            default:
                cout << "Invalid choice! Try again.\n";
        }
    }
```

```

    } while (choice != 4);

    return 0;
}

```

8. Implement sorting algorithms such as quick sort, merge sort, and heap sort, and compare their performance.

9. Create a binary search tree (BST) in C++ and perform insert, delete, and search operations.

```

#include <iostream>
using namespace std;

// Structure of a BST node
struct Node {
    int data;
    Node* left;
    Node* right;
};

// Function to create a new node
Node* createNode(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// Insert a value in BST
Node* insert(Node* root, int value) {
    if (root == NULL)
        return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

```

```

}

// Search a value in BST
bool search(Node* root, int value) {
    if (root == NULL)
        return false;

    if (root->data == value)
        return true;
    else if (value < root->data)
        return search(root->left, value);
    else
        return search(root->right, value);
}

// Find minimum value node (used in delete)
Node* findMin(Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

// Delete a value from BST
Node* deleteNode(Node* root, int value) {
    if (root == NULL)
        return root;

    if (value < root->data)
        root->left = deleteNode(root->left, value);
    else if (value > root->data)
        root->right = deleteNode(root->right, value);
    else {
        // Node with one or no child
        if (root->left == NULL) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        else if (root->right == NULL) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        // Node with two children
        Node* temp = findMin(root->right);

```

```

        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

```

// Inorder Traversal (prints BST in sorted order)

```

void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

```

// Main function

```

int main() {
    Node* root = NULL;

    // Insert values
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);

    cout << "Inorder Traversal: ";
    inorder(root);

    cout << "\nSearch 40: ";
    if (search(root, 40))
        cout << "Found";
    else
        cout << "Not Found";

    cout << "\nDeleting 20...\n";
    root = deleteNode(root, 20);

    cout << "Inorder after deletion: ";
    inorder(root);

    return 0;
}

```

10. Implement Depth First Search (DFS) algorithm for a graph using adjacency matrix representation in C++.

```
#include <iostream>
using namespace std;

int n;          // Number of vertices
int adj[10][10]; // Adjacency matrix
int visited[10]; // To mark visited vertices

// DFS function
void DFS(int v) {
    cout << v << " "; // Print current vertex
    visited[v] = 1;     // Mark as visited

    for (int i = 0; i < n; i++) {
        if (adj[v][i] == 1 && visited[i] == 0) {
            DFS(i); // Recursive call
        }
    }
}

int main() {
    int start;

    cout << "Enter number of vertices: ";
    cin >> n;

    cout << "Enter adjacency matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> adj[i][j];
        }
    }

    // Initialize visited array
    for (int i = 0; i < n; i++) {
        visited[i] = 0;
    }

    cout << "Enter starting vertex: ";
    cin >> start;
```

```
    cout << "DFS Traversal: ";  
    DFS(start);  
  
    return 0;  
}
```